



Week 03 – Building API with Database Integration

Database Integration Fundamentals

Connecting APIs to relational databases enables data persistence, efficient data management, secure access, and reliable operations.

The Problem with In-Memory Data



The Problem

Data stored in memory is **temporary** — lost whenever the server restarts. Managing larger datasets or concurrent users becomes increasingly difficult without a dedicated system.

The Solution

A database provides persistent, structured storage — organized for easy access, management, and updates. Relational databases store data in tables (e.g., books with title, author, ID) with clearly defined relationships.

Why Connect APIs to Databases?



Persistence

Data survives server restarts



Scalability

Handles growing datasets efficiently



Data Integrity

Enforces relationships and constraints



Concurrency

Multiple users, no conflicts

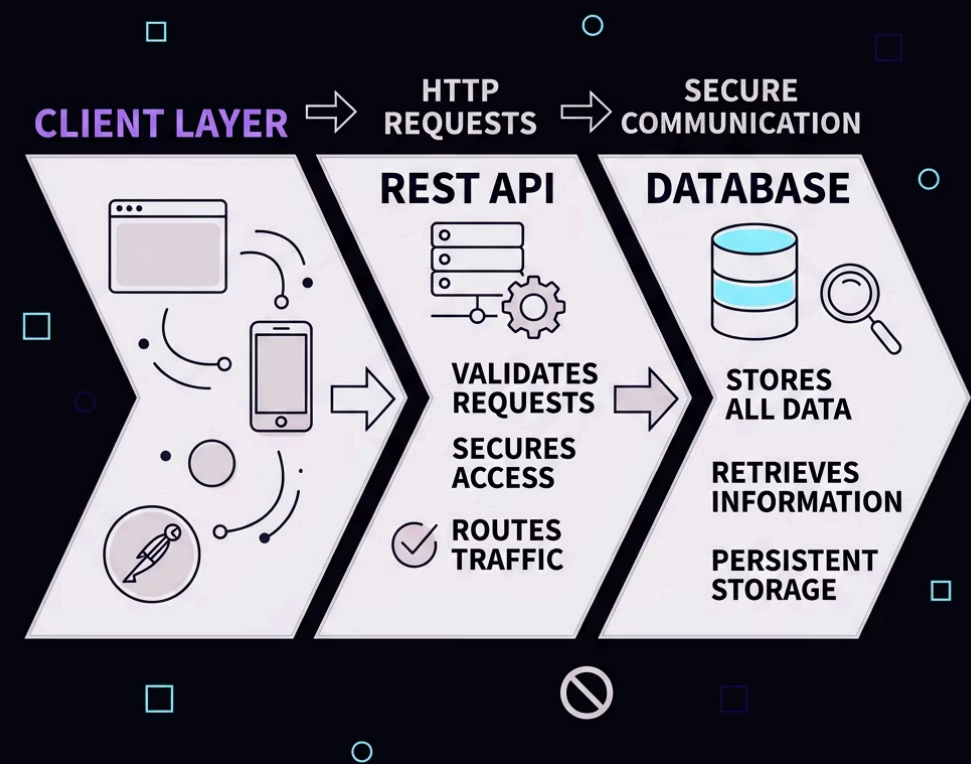


Security

Controlled, private data access

Using our example of **Books API**, integrating with **MSSQL** means books persisting beyond restarts, support efficient querying, and can be updated or removed reliably.

REST API as an Intermediary

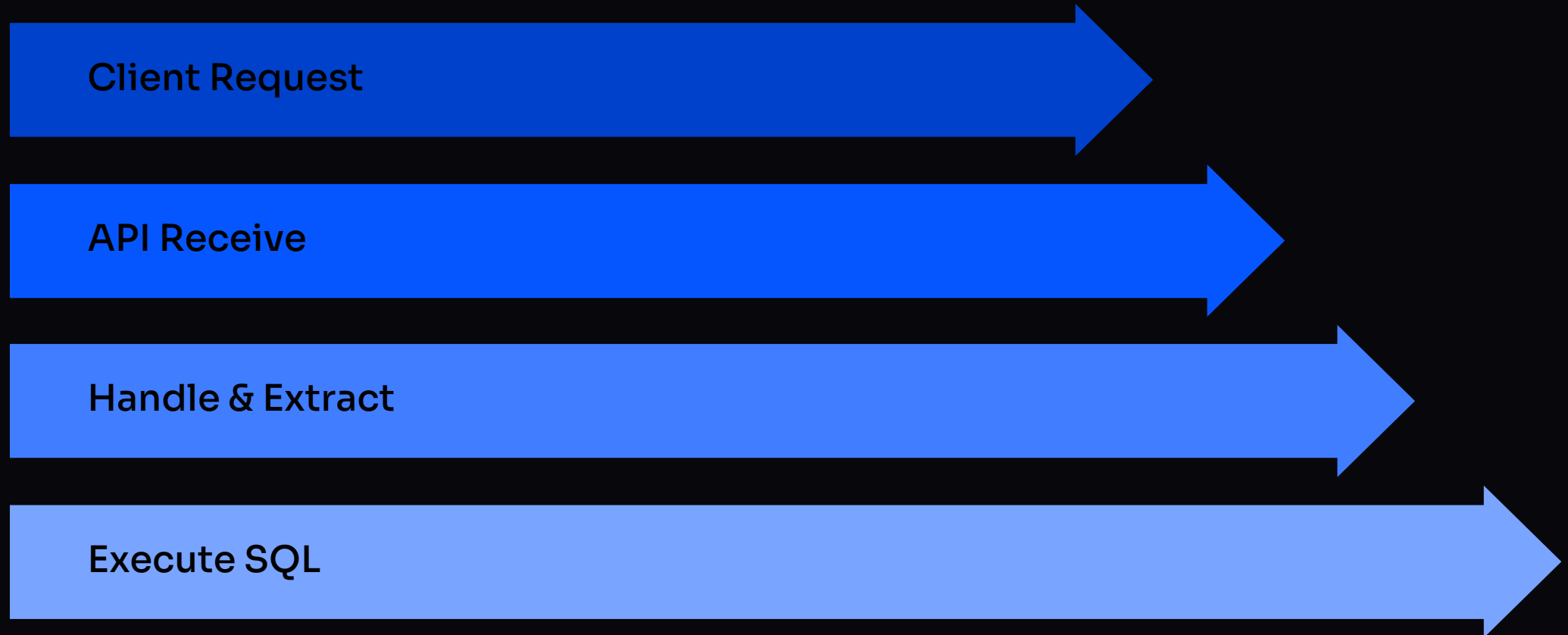


HTTP Methods → CRUD Operations

POST /books	CREATE (INSERT)
GET /books	READ (SELECT all)
GET /books/:id	READ (SELECT one)
PUT /books/:id	UPDATE
DELETE /books/:id	DELETE

The API validates requests, enforces security, and shields the client from database complexity.

REST API – Database Interaction Flow



The client only interacts with the API — never the database directly. The API handles validation, SQL execution, error management, and JSON formatting before returning an appropriate HTTP status code.

Node.js & MSSQL Integration

What is Node.js?

A JavaScript runtime built on Chrome's V8 engine, widely used for server-side applications.

What is MSSQL?

Microsoft SQL Server — a robust relational database management system.

How They Connect

The `mssql` npm package bridges Node.js and SQL Server for queries, transactions, and secure data operations.

- ① Focus on understanding the **why** and **how** — hands-on coding will happen during the in-class practical session.



Setting Up: Environment & Connection

Steps 1 & 2: Setup

1 Install Node.js

Enable TCP/IP on MSSQL Server for remote connections. (See GitHub instruction screenshots)

2 Install mssql package

```
npm install mssql
```

Above line to be entered in your MS Visual Studio Code Terminal, when you "cd" into the workspace.

Step 3: Connect to Database

```
const sql = require('mssql');
const config = {
  user: 'your_username',
  password: 'your_password',
  server: 'your_server',
  database: 'your_database',
  options: {
    encrypt: true,
    trustServerCertificate: true
  }
};

async function connectToDatabase() {
  try {
    await sql.connect(config);
    console.log('Connected to MSSQL');
  } catch (err) {
    console.error('Connection failed:', err);
  }
}
```

CRUD Operations Examples in Node.js + MSSQL

1

Create

```
await sql.query("INSERT INTO users  
(name, email)  
VALUES ('John Doe',  
'john@example.com')");
```

2

Read

```
await sql.query(  
"SELECT * FROM users");
```

3

Update

```
await sql.query("UPDATE users  
SET email='john.new@example.com'  
WHERE name='John Doe'");
```

4

Delete

```
await sql.query(  
"DELETE FROM users  
WHERE name='John Doe'");
```


Practical Considerations



Error Handling

Implement robust handling for connection issues and query failures to provide meaningful feedback.



Security

Store credentials in **environment variables** — never hardcode sensitive information.



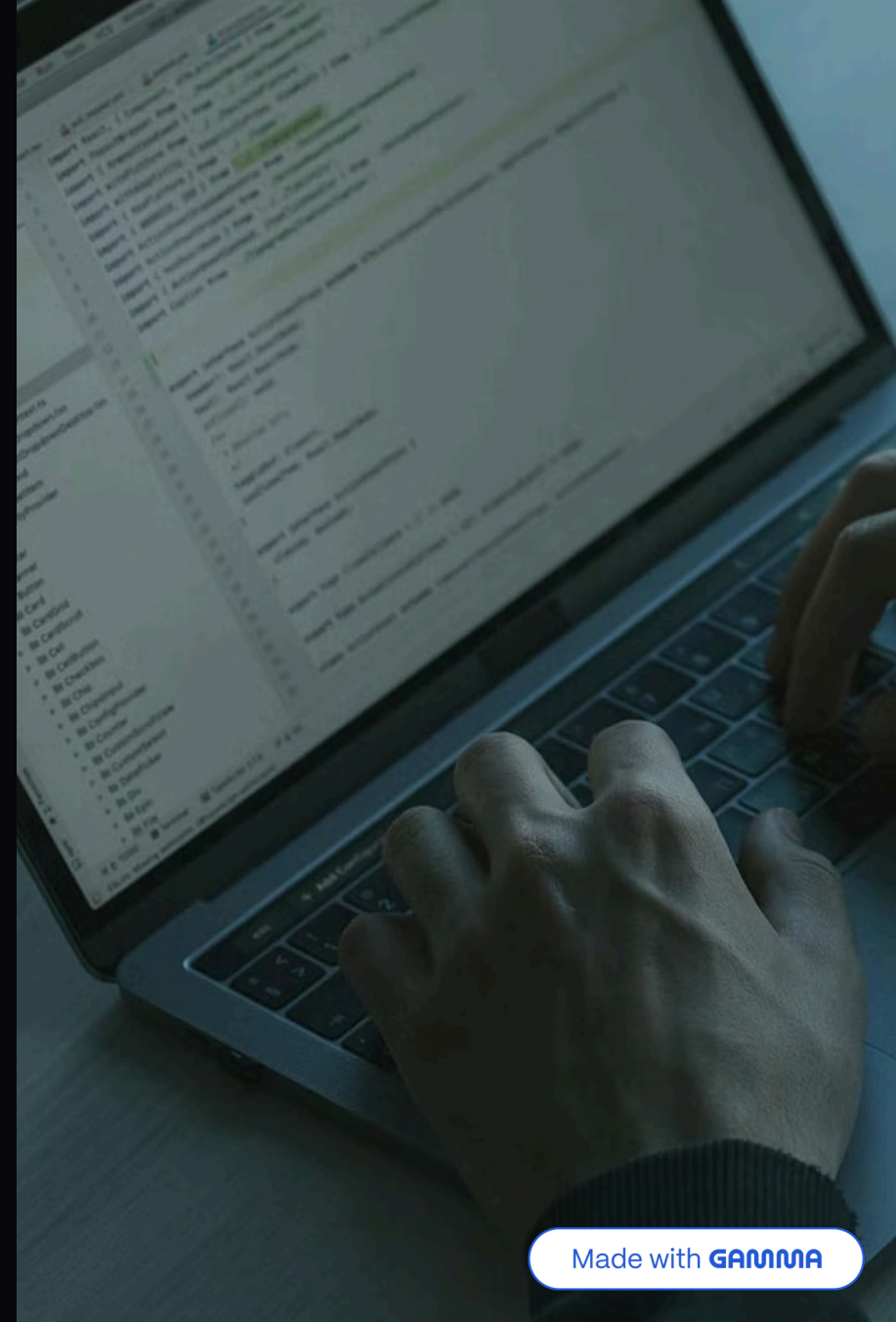
SQL Injection Prevention

Use **parameterized queries** so user inputs are treated as data, not executable code.



Data Validation

Validate all incoming data to maintain integrity and prevent consistency issues.



Summary & Key Takeaways

01

Setup Environment

Install Node.js and configure MSSQL with TCP/IP enabled.

02

Use the mssql Package

Connect Node.js applications to MSSQL databases via npm.

03

Perform CRUD Operations

Create, read, update, and delete records using SQL statements.

04

APIs as Intermediaries

REST APIs simplify client-database interaction and enhance security.

05

Apply Best Practices

Error handling, parameterized queries, and environment variables are essential for robust, secure apps.

- ✔ Persistent databases are vital for reliable data management. Keep building on this knowledge to create **secure, efficient, and scalable** applications.